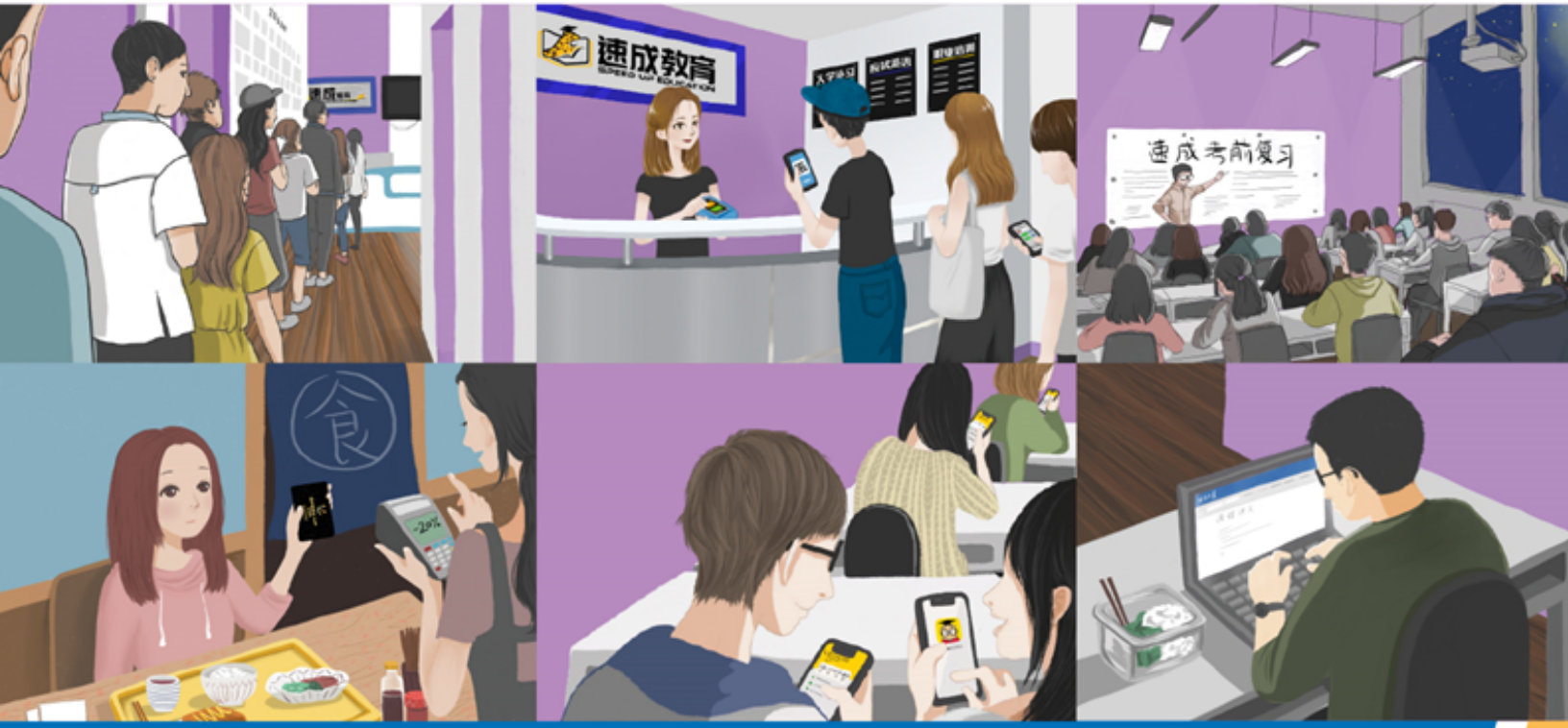
速成教育
SPEED UP EDUCATION



APSC 160

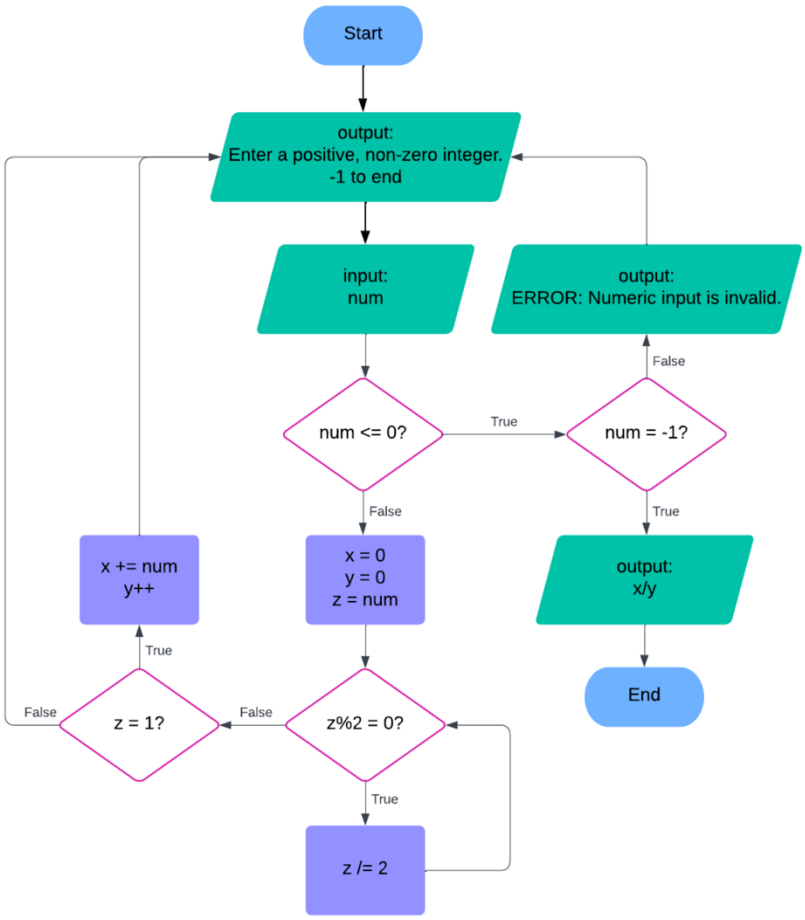
Introduction to Computation in Engineering Design

导师: **VC**

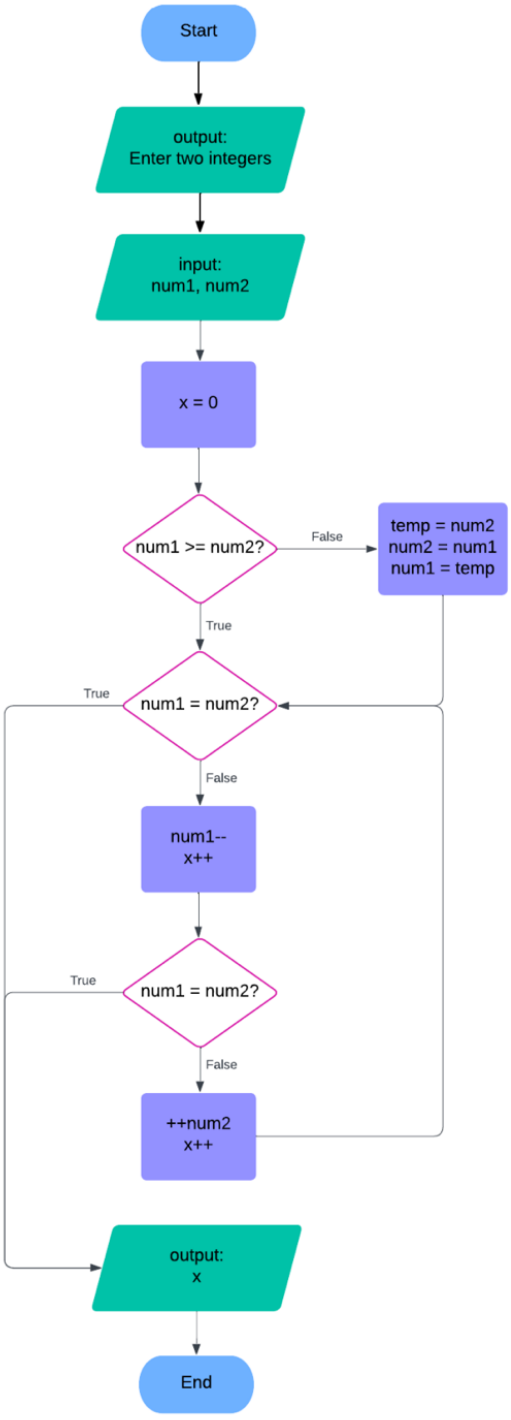
UBC Midterm 1 Review 2026/10/3

Question1: Control flow chat

a) Explain in plain English what the purpose of the function is. Simplify the explanation as much as possible. Also Implement the program in C with better variable names.



b) Explain in plain English what the purpose of the function is. Simplify the explanation as much as possible. Also Implement the program in C with better variable names.



c) Consider the following program specification:

Repeatedly:

- prompt the user for a time and read it from the keyboard
- compute the position p of the particle at that time t using the formula: p = 10sin(t)
- print on the screen the position of the particle to 2 decimal places

until the user enters a time that is negative. The program must terminate as soon as a negative time has been read from the keyboard - it must not compute the particle's position at this time or print that position on the screen.

Draw a flowchart that represents a solution to this problem.

Question 2: Code Tracing Practice

Write down the output when running the following programs.

```c
#include <stdio.h>
int main(void){
    int i = 0, j = 0;
    while (i++ < 2){
        int k = i;
        do{
            j += k;
            k--;
        }while (k > 1);
    }
    printf("%d %d\n", i, j);
    return 0;
}
```

Output
>

```c
#include <stdio.h>
int main(void){
    int i = 0, j = 0, p = 0;
    while (i < 3){
        j = 0;
        do{
            p++;
        }while (j++ < i);
        i++;
    }
    printf("%d %d %d\n", i, j, p);
    return 0;
}
```

Output
>

```c
#include <stdio.h>
int main(void){
    int a = 1, b = 2;
    int i = 0;
    while (i < 3){
        a += b++ + ++a;
        i++;
    }
    printf("%d %d\n", a, b);
    return 0;
}
```

Output
>

```c
#include <stdio.h>
int main(void){
    int i = 3;
    int j;
    while (i >= 0){
        j = 0;
        while (j < 3){
            if (j == i) printf("X");
            else if (j < i) printf("%d", j);
            else printf("%d", i);
            j++;
        }
        printf("\n");
        i--;
    }
    return 0;
}
```

Output

```
>
```

```c
#include <stdio.h>
int main(void){
    int i = 0;
    while (i < 3){
        int t = 3;
        do{
            if (i == t) printf("=");
            else if (i > t) printf(">");
            else printf("<");
            t--;
        }while (t >= 1);
        printf("\n");
        i++;
    }
    return 0;
}
```

Output

```
>
```

```c
#include <stdio.h>
int main(void){
    int i = 0, ones = 0;
    while (i < 4) {
        int j = 0;
        do{
            if ( (i + j) % 2 == 0 ) {
                printf("1");
                ones++;
            } else {
                printf("0");
            }
            j++;
        } while (j <= i);
        printf(" [%d]\n", ones);
        i++;
    }
    return 0;
}
```

Output

```
>
```

```c
#include <stdio.h>
int main(void){
    int i = -1, j, k, mark = 0, sum = 0;

    while (i++ < 2) {
        k = i;
        j = -1;
        int seenX = 0;

        do {
            int next  = j + 1;
            int isPeak = (next == k);

            if ((k % 2 != 0) && !isPeak) {
                printf("%d", k);
                sum += k;
            } else if (isPeak && !seenX) {
                printf("X");
                mark++;
                seenX = 1;
            } else {
                printf("%d", next);
            }

            j++;
        } while (j < k);

        if ((sum + mark) % 2 == 0) printf("|E\n");
        else                       printf("|O\n");
    }

    printf("[%d,%d]\n", sum, mark);
    return 0;
}
```

Output

```
>
```

Question3: Loops

一定要聽懂上課講的 TT1_while_template.c 和 TT1_do_while_template.c, 考試題目都是類似的格式!

a) Write a program that:

- repeatedly prompts the user for his/her age until an age between 1 and 125 inclusive (i.e., including 1 and 125) is entered. If the user enters an age that is not valid (i.e., not in the specified range), your program must print an error message before prompting for another value.
- prints a message on the screen to indicate the cost of a ticket (to 2 decimal places) for a tour bus as indicated in the table below:

age of tourist	cost of ticket
child - under 18	$13.45
adult – 18 to 64 inclusive	$21.95
senior – 65 or older	$18.95

b) Write a program that prompts the user to repeatedly enter pairs of positive integers until at least one of the numbers in a pair is less than or equal to zero. The program must then print out the largest of all the first numbers entered in the pairs and the smallest of all the second numbers entered – *excluding* the last pair entered (i.e., the one that contains a value that is less than or equal to zero). To simplify the program, you can assume that the user enters at least one pair of positive integers. Here are a couple of sample runs:

```
Please enter a sequence of pairs of positive integers.
Terminate by entering at least one non-positive value
in a pair.
2 6
7 1
3 5
4 4
8 2
4 -1
Max of first numbers: 8
Min of second numbers: 1


Please enter a sequence of pairs of positive integers.
Terminate by entering at least one non-positive value
in a pair.
4 5
7 6
-1 4
Max of first numbers: 7
Min of second numbers: 5
```

c)

Complete the program provided below so that it prompts the user for a series of positive integers. The user enters the value -1 to indicate that there is no more data to enter. The function must print the smallest and largest of the integer values entered by the user (excluding the value -1). If the first value entered by the user is -1 (in other words, no positive integers are entered), your program must print an error message and terminate. Here are a couple of sample runs:

Sample run #1

```
Enter a series of positive integers,
terminated by -1:
-1
ERROR: no data entered!
Press any key to continue...
```

Sample run #2

```
Enter a series of positive integers,
terminated by -1:
4
2
10
3
-1
Minimum value: 2, maximum value: 10
Press any key to continue...
```

```
#include <stdio.h>
#include <stdlib.h>

#define SENTINEL -1

int main( void ) {
```

d) Write a program in C that computes and returns (as a double value) the sum of the following series of numbers: $1 + x + x^2 + x^3 + x^4 + \ldots + x^{N-1} + x^N$ where the integer N and the double x are the inputs. Here are some examples:

N	x	value returned	explanation
0	3.3	1	always 1 for N=0
3	1	4	$1 + 1 + 1^2 + 1^3$
2	.5	1.75	$1 + 0.5 + 0.5^2$

Note that your function is not expected to print the explanation – this has been provided only to help you understand what the function is supposed to do. For full marks on this question, do not use the pow math library function or any function that you write yourself that computes one number raised to the power of another.

e) The following formula provides an approximation to ln(x) for values of x that are close to 1:

$$(x-1)-\frac{(x-1)^2}{2}+\frac{(x-1)^3}{3}-\frac{(x-1)^4}{4}+\cdots$$

Note that the pattern you see here repeats indefinitely – only the first four terms are shown. In general, the nth term is given by the formula:

$$(-1)^{n+1}\frac{(x-1)^n}{n}$$

Before you go any further, convince yourself that if you plug 1 in for n, you get the first term in the formula above and if you plug 2 in for n, you get the second term, etc.

Write a program that prompts the user for a value of x and that produces an approximation to ln(x) using the formula above and prints it on the screen. Assuming the value of x is such that $0 < x < 2$, the size of successive terms gets smaller and smaller. You must keep adding terms until the size of the term to be added, in absolute value, drops below 1.0×10^{-6}. Your program must also print the number of terms used to compute the approximation. Here is a sample run:

```
Please enter value of x (0 < x < 2): 1.25

Approximation to ln(1.250000) is 0.223143
- computed using 8 terms.
```

You might find the following functions from the math library useful:

```
// returns base to the power of exp: (base)^exp
double pow(double base, double exp);

// returns the absolute value of num
double fabs(double num);
```

f)

Complete the program below so that it:

- prompts the user for the number of rows of data (**nRows**) in a table of values
- prompts the user for the number of columns of data (**nCols**) in a table of values (excluding the first)
- prints a table of values of the function **sin(cx)** for increasing values of **c** (1, 2, 3, ... determined by the number of columns) and for each of the following input values:
  π/nRows, 2π/nRows, 3π/nRows …, π
  as illustrated below
- prints the maximum value in the table and the value of **x** that produced that value, also as illustrated below; note that if the maximum value occurs more than once, your program may print any one of the corresponding values of **x** (but only one)

nRows: 5, nCols: 1

```
x       sin(x)
0.628   0.5878
1.257   0.9511
1.885   0.9511
2.513   0.5878
3.142   0.0000
Maximum output value of 0.9511
occurs at x = 1.885
```

nRows: 8, nCols: 3

```
x       sin(x)   sin(2x)  sin(3x)
0.393   0.3827   0.7071   0.9239
0.785   0.7071   1.0000   0.7071
1.178   0.9239   0.7071  -0.3827
1.571   1.0000   0.0000  -1.0000
1.963   0.9239  -0.7071  -0.3827
2.356   0.7071  -1.0000   0.7071
2.749   0.3827  -0.7071   0.9239
3.142   0.0000  -0.0000   0.0000
Maximum output value of 1.0000
occurs at x = 0.785
```

Note that the tables above represent only two possible outputs – your program must be able to handle any reasonable number of rows and columns input by the user.

Pay particular attention to the number of decimal places to which the output is printed and the alignment of data in each column – your program must produce output exactly as illustrated above. **However, the column headers (on the first row of output) shown in bold face text are for your reference only and must not be printed by your program.**

```
#include <stdio.h>
#include <math.h>
#define PI acos (-1.0)

int main(void) {

}
```

g) Write a program that prompt the user for a positive integer representing a number of rows and draw a ramp of '*' characters with the given number of rows.

For example:

Number of rows:	1	2	3
Ramp drawn:	*	* ***	* *** *****

g) Similar to Question f) but now print a pyrmind instead.

For example:
If the number of row is 5, display:

```
    *
   * *
  *   *
 *     *
*********
```

h)

Write a program to evaluate the quality of multiple coffee-roasting batches. The program first reads an integer B (the number of batches, with B ≥ 1). For each batch `i` from 1 to B, read two integers: temptemptemp (the roasting temperature in Celsius, between 150 and 250 inclusive) and timetimetime (the roasting time in seconds, between 300 and 1200 inclusive). If either value is outside its range, immediately print an error and re-enter that batch until both values are valid. For each batch, compute its quality score using the formula

$$score = \frac{(temp - 200)^2}{400} + \frac{(time - 600)^2}{3600}$$

A smaller score means the batch is closer to the target; a batch **passes** if score≤1.0 and **fails** otherwise. After all batches have been processed, print for each batch its score (to three decimal places) and PASS/FAIL; then report the index of the best batch (the one with the smallest score, breaking ties by choosing the smaller index), the number of passes and the pass ratio (to two decimal places), and the average score across all batches (to three decimal places). Do not use arrays (compute and aggregate on the fly) and do not call `pow`. *Hint:* you may use `DBL_MAX` from `<float.h>` to obtain the largest representable `double`.

咖啡烘焙质量管制

1. 读入整数 `B` (number of batches)（批次数，必须 ≥ 1）。若无效则重新输入。
2. 对每个批次 `i`（1..B）：
   - 读入 `temp`（烘焙温度，摄氏 150–250 的整数）与 `time`（烘焙时间，秒，300–1200 的整数）。若任一数值不在范围，立即打印错误并重新输入该批次直到有效。
   - 定义质量指标：

     $$score = \frac{(temp-200)^2}{400} + \frac{(time-600)^2}{3600}$$

     分数越小越接近目标。若 `score ≤ 1.0` 则 **PASS**，否则 **FAIL**。

   - 输出 各批次的 `score`（打印到小数点后 3 位）与 PASS/FAIL
3. 全部批次处理完成后，输出：
   - 最佳批次编号（分数最小；若并列取编号较小者）
   - PASS 的数量与比例（比例打印到小数点后 2 位）
   - 所有 score 的平均值（小数点后 3 位）。

i)

```
A water tank has a maximum capacity of 500 litres.
The tank initially contains 200 litres.

Repeatedly ask the user to enter a flow value.
    Positive value    -> water enters the tank
    Negative value    -> water leaves the tank
    0                 -> terminate the simulation

Update the water level after every input.
The water level must never be allowed to go below 0 L
or above 500 L.

If an input would cause the tank to exceed 500 L:
     display "Overflow prevented"
     set the water level to 500 L.

If an input would cause the tank to go below 0 L:
     display "Tank empty"
     set the water level to 0 L.

After every valid update, classify the tank:
     level < 100 L          -> LOW
     100 <= level < 400 L    -> NORMAL
     level >= 400 L         -> HIGH

When 0 is entered, display:
1) Final water level
2) Highest water level reached
3) Lowest water level reached
4) Number of overflow events
5) Number of empty-tank events
```

j)

A machine contains a temperature sensor.
Repeatedly prompt the user for a temperature in degrees Celsius.

Enter -999 to terminate.

Valid temperatures are between -50 C and 150 C inclusive.
Invalid temperatures must display an error message and must
not be included in any calculations.

For every valid temperature:
    Convert Celsius to Fahrenheit:
        F = C * 9.0 / 5.0 + 32

    Classify the machine:
        C < 20   -> COLD
        20 <= C < 80   -> NORMAL
        80 <= C < 120   -> HOT
        C >= 120   -> CRITICAL

    If three CRITICAL readings are entered consecutively,
    immediately display:

        "EMERGENCY SHUTDOWN"

    and terminate the program.

Otherwise, continue until -999 is entered.

At the end display:

1) Number of valid readings
2) Number of invalid readings
3) Average temperature in Celsius
4) Highest temperature
5) Lowest temperature
6) Number of HOT readings
7) Number of CRITICAL readings
8) Whether an emergency shutdown occurred

If no valid temperatures were entered:
     "Error: no valid data entered!"